
Escuela de Ingeniería de Sistemas

Desarrollo de Aplicaciones de Última Generación

Avance I

Estudiante

Raquel Sanabria Ramírez

Josué Sánchez Miranda

Adriana Barboza Oviedo

Profesor

Andrés Ulloa Brenes

Cartago, Costa Rica

Marzo, 2026

Contenido

Proyecto Final	3
¿Qué se va a hacer?	3
¿Quién lo va a hacer?	3
¿Cómo se va a hacer?	3
¿Por qué se va a hacer?	4
¿Para qué se va a hacer?	4
¿Cuándo se va a hacer?	4
Alcance preliminar	5
Beneficios esperados	6
Planificación del proyecto	7
Diseño de Pantallas	7
4. Botón para reconectar con PostgreSQL	9

Proyecto Final

Proyecto: Aplicación en Tiempo Real (Chat en Línea o Envío de Facturas) con Broker de Mensajería y Almacenamiento en Base de Datos

¿Qué se va a hacer?

Descripción General Se debe desarrollar una solución de comunicación en tiempo real entre dos aplicaciones (una en Java y otra en Python), ambas con interfaz gráfica. El alcance funcional puede ser: (A) un chat en línea, o (B) el envío de facturas entre dos puntos. La mensajería se implementará mediante un broker (RabbitMQ o Kafka) y toda interacción (mensajes o facturas) deberá persistirse en PostgreSQL y Microsoft SQL Server para consulta y auditoría

¿Quién lo va a hacer?

El proyecto será liderado por el grupo III.

¿Cómo se va a hacer?

Según el alcance elegido (Chat o Facturas) y la tecnología de mensajería (RabbitMQ o Kafka), incluya en la configuración:

RabbitMQ: crear exchange/colas, bindings y políticas (DLQ opcional). Documentar usuarios/vhosts/permisos.

Kafka: crear topic(s), particiones, replication factor (si aplica), y documentar bootstrap servers y configuración de seguridad.

- Instalación del motor de mensajería (RabbitMQ o Kafka) en el servidor o en una máquina local.
- Configuración de parámetros básicos de funcionamiento (puertos, autenticación, etc.).
- Implementación de autenticación y autorización para la protección del sistema de mensajería.
- Establecimiento de conexiones seguras (SSL/TLS) si es necesario.
- Instalación y configuración de **PostgreSQL** y **Microsoft SQL Server (MSSQL)** como sistemas de gestión de bases de datos.

Creación de tablas según el alcance elegido:

Si eligió Chat en línea (mensajes):

PostgreSQL:

```
CREATE TABLE messages (  
  id SERIAL PRIMARY KEY,  
  sender VARCHAR(100),  
  receiver VARCHAR(100),  
  message TEXT,  
  status VARCHAR(30) DEFAULT 'sent',  
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

MSSQL:

```
CREATE TABLE messages (  
  id INT IDENTITY(1,1) PRIMARY KEY,  
  sender NVARCHAR(100),  
  receiver NVARCHAR(100),  
  message NVARCHAR(MAX),  
  status NVARCHAR(30) DEFAULT 'sent',  
  timestamp DATETIME DEFAULT GETDATE()  
);
```

- Verificación de la operatividad con pruebas de envío y recepción en tiempo real (mensajes o facturas, según el alcance).

¿Por qué se va a hacer?

Se realizará con la finalidad de desarrollar habilidades en el desarrollo para que los estudiantes puedan tener conocimiento de desarrollo en colas en tiempo real, el cual permitirá entender cómo funciona un sistema de mensajería para el envío de mensajes en tiempo real.

¿Para qué se va a hacer?

Este sistema busca facilitar la comunicación entre 2 aplicaciones diferentes y permitir el monitoreo en tiempo real, generando así un impacto positivo en la calidad del servicio.

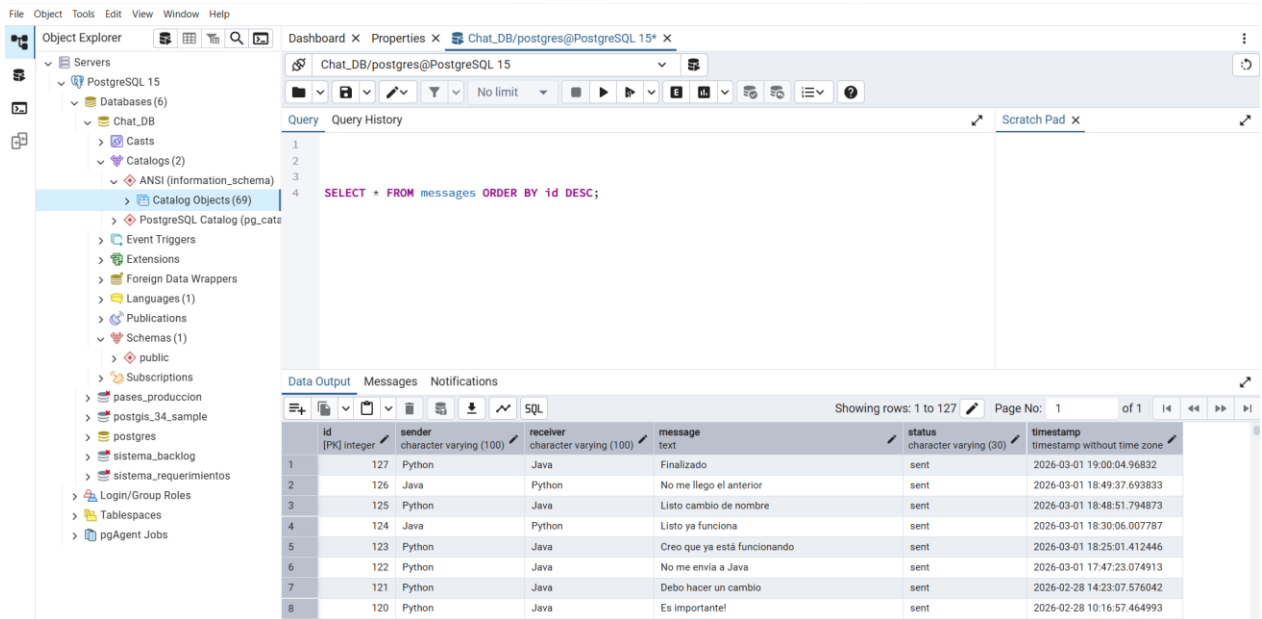
¿Cuándo se va a hacer?

El proyecto se ejecutará desde el 16 de enero hasta el 24 de abril de 2026, con una duración total de 15 semanas. Cada fase cuenta con fechas definidas y puntos de control para asegurar el cumplimiento de los objetivos establecidos.

Alcance preliminar

El proyecto contempla el desarrollo e implementación de un sistema digital integral de envío de mensajes que brinde una solución completa y escalable a las necesidades actuales en desarrollo para sistemas que deberán realizar envío de mensajes en tiempo real. El sistema incluirá las siguientes funcionalidades clave:

- Reenvío de mensajes en tiempo real
- Es un chat en línea
- Se utilizará la base de datos denominada Chat_DB en Postgress



The screenshot shows the DBeaver database management tool interface. On the left, the 'Object Explorer' pane displays the database structure for 'PostgreSQL 15', including 'Databases (6)', 'Catalogs (2)', 'ANSI (Information_schema)', 'Catalog Objects (69)', 'PostgreSQL Catalog (pg_catalog)', 'Event Triggers', 'Extensions', 'Foreign Data Wrappers', 'Languages (1)', 'Publications', 'Schemas (1)', 'public', 'Subscriptions', 'pases_produccion', 'postgres_34_sample', 'postgres', 'sistema_backlog', 'sistema_requerimientos', 'Login/Group Roles', 'Tablespaces', and 'pgAgent Jobs'. The main pane shows a query editor with the following SQL query:

```
SELECT * FROM messages ORDER BY id DESC;
```

Below the query editor, the 'Data Output' pane displays the results of the query in a table format. The table has 8 columns: 'id [PK] integer', 'sender character varying (100)', 'receiver character varying (100)', 'message text', 'status character varying (30)', and 'timestamp timestamp without time zone'. The results show 8 rows of data, ordered by 'id' in descending order.

id [PK] integer	sender character varying (100)	receiver character varying (100)	message text	status character varying (30)	timestamp timestamp without time zone
1	127	Python	Java	Finalizado	2026-03-01 19:00:04.96832
2	126	Java	Python	No me llevo el anterior	2026-03-01 18:49:37.693833
3	125	Python	Java	Listo cambio de nombre	2026-03-01 18:48:51.794873
4	124	Java	Python	Listo ya funciona	2026-03-01 18:30:06.007787
5	123	Python	Java	Creo que ya está funcionando	2026-03-01 18:25:01.412446
6	122	Python	Java	No me envia a Java	2026-03-01 17:47:23.074913
7	121	Python	Java	Debo hacer un cambio	2026-02-28 14:23:07.576042
8	120	Python	Java	Es importante!	2026-02-28 10:16:57.464993

- La comunicación se realiza Python y Java a través de RabbitMQ
- Estructura de la tabla PostgreSQL

Estructura de tabla messages (PostgreSQL)

Columna	Tipo	Descripción	Notas
id	SERIAL PRIMARY KEY	Auto-incremental	
sender	VARCHAR(100)	Ej: Python / Java	
receiver	VARCHAR(100)	Destinatario	
message	TEXT	Contenido del mensaje	
status	VARCHAR(30)	Default: sent	
timestamp	TIMESTAMP	NOW() al insertar	

Data Output Messages Notifications						
Showing rows: 1 to 127 Page No: 1 of 1						
	id [PK] integer	sender character varying (100)	receiver character varying (100)	message text	status character varying (30)	timestamp timestamp without time zone
1	127	Python	Java	Finalizado	sent	2026-03-01 19:00:04.96832
2	126	Java	Python	No me lleo el anterior	sent	2026-03-01 18:49:37.693833
3	125	Python	Java	Listo cambio de nombre	sent	2026-03-01 18:48:51.794873
4	124	Java	Python	Listo ya funciona	sent	2026-03-01 18:30:06.007787
5	123	Python	Java	Creo que ya está funcionando	sent	2026-03-01 18:25:01.412446
6	122	Python	Java	No me envía a Java	sent	2026-03-01 17:47:23.074913
7	121	Python	Java	Debo hacer un cambio	sent	2026-02-28 14:23:07.576042
8	120	Python	Java	Es importante!	sent	2026-02-28 10:16:57.464993
9	119	Java	Python	Listo	sent	2026-02-25 20:19:20.95297

Beneficios esperados

- Reducción significativa en el tiempo para realizar el proceso de envío de mensajes.
- Mayor transparencia, trazabilidad y control en cada etapa del proceso.
- Acceso en tiempo real a datos
- Comunicación entre bases de datos de Java a Python y de Python a Java

Planificación del proyecto

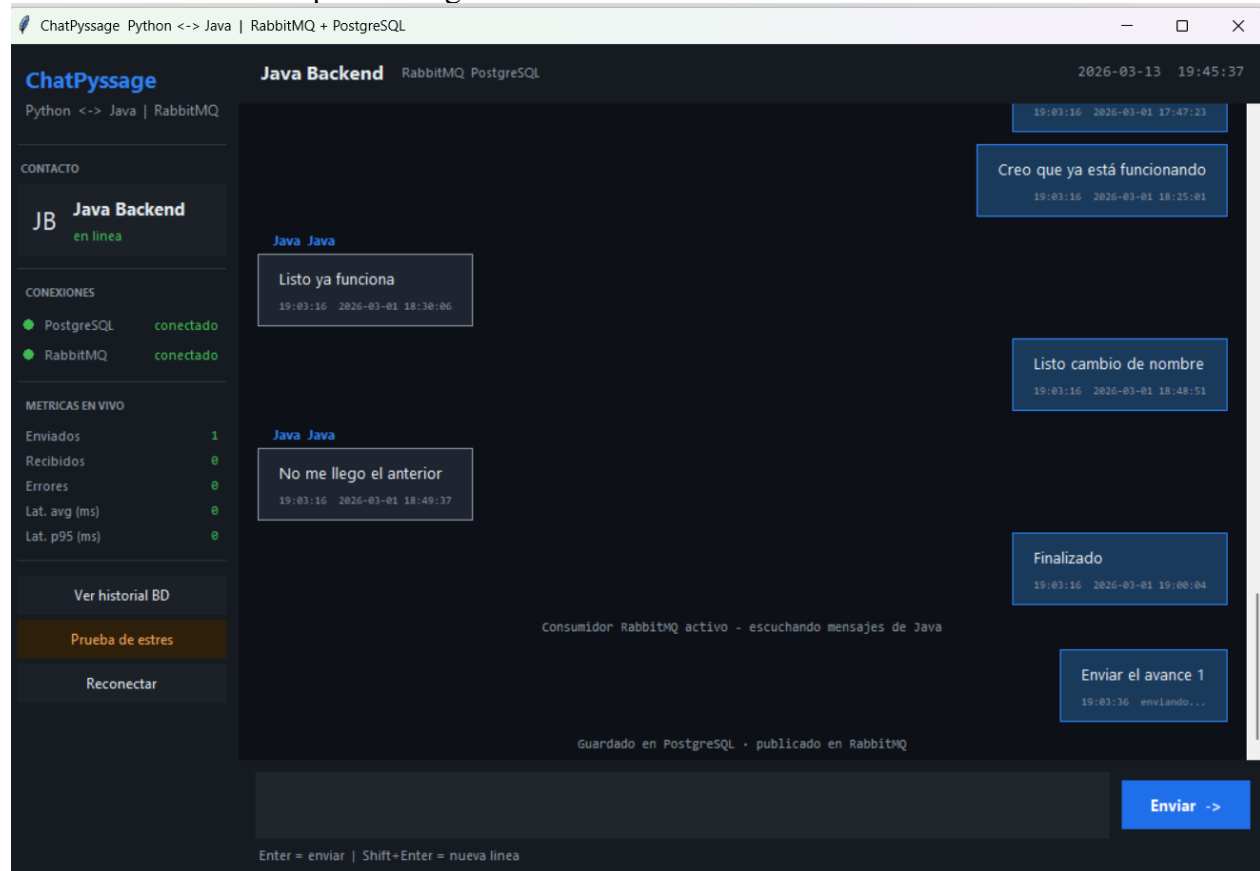
Fase	Duración	Fecha de Inicio	Fecha de Finalización
Análisis de requisitos	1 semana	enero	24 enero
Diseño del sistema	3 semanas	25 enero	14 febrero
Desarrollo	6 semanas	15 febrero	14 marzo
Pruebas	2 semanas	15 marzo	28 marzo
Capacitación e implementación	2 semanas	29 marzo	11 abril
Seguimiento puesto en producción	1 semana	12 abril	18 abril

Diseño de Pantallas

Para El envío de mensajes desde PostgreSQL se creó la pantalla de escritorio

1. El nombre se definió como ChatPyssage el cual se define como:

- ✓ PY por Python
- ✓ SSAGE por Message



2. Tiene un botón para ver Historial de la BD

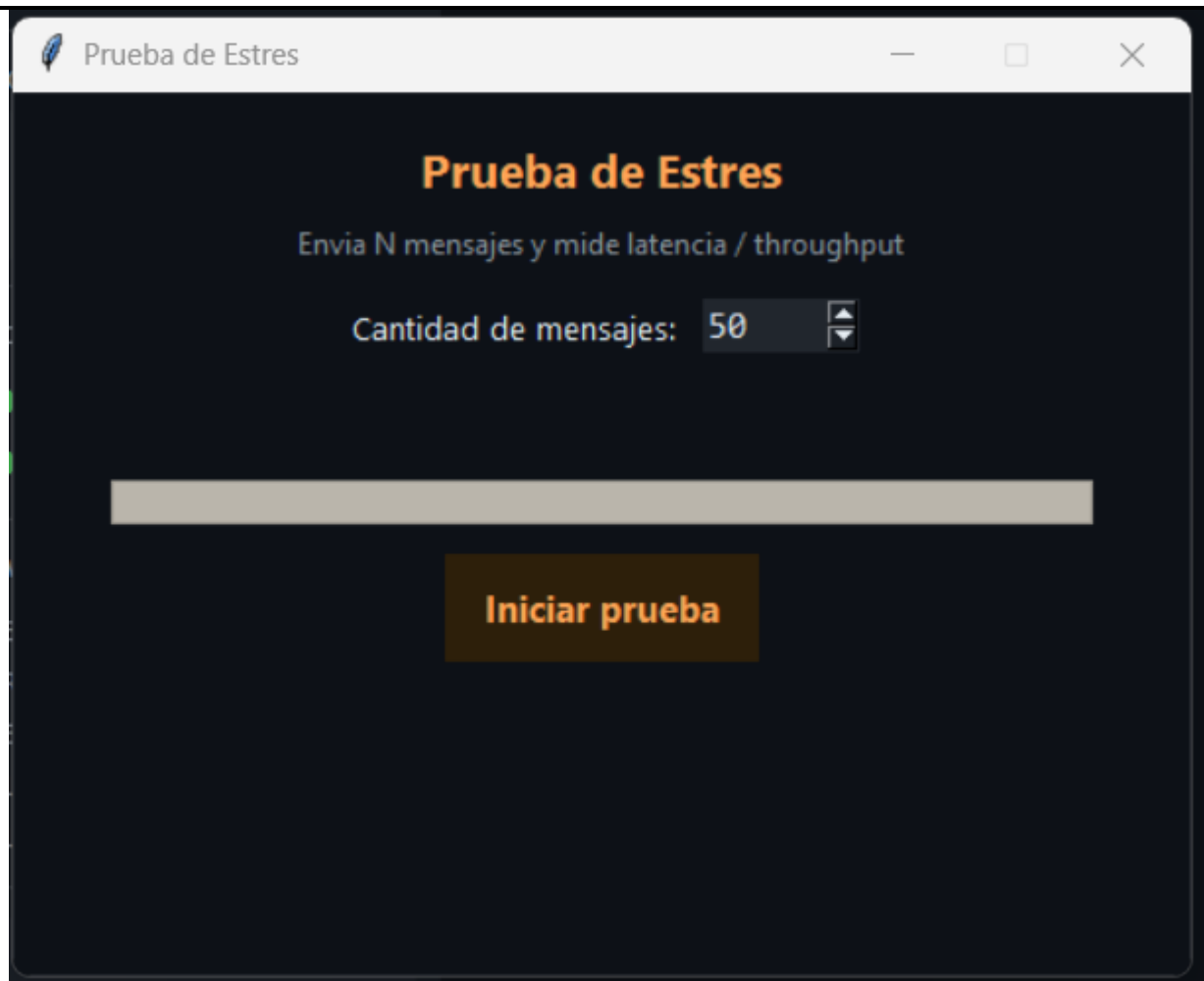
Historial - PostgreSQL

Historial de mensajes en PostgreSQL

100 registros encontrados

ID	SENDER	RECEIVER	MESSAGE	STATUS	TIMESTAMP
29	Python	Java	[stress] msg_0012 @ 2026-02-25T18:16:20.7	sent	2026-02-25 18:16:20.846
30	Java	Python	[stress] msg_0012 @ 2026-02-25T18:16:20.7	sent	2026-02-25 18:16:20.849
31	Python	Java	[stress] msg_0013 @ 2026-02-25T18:16:20.8	sent	2026-02-25 18:16:20.949
32	Java	Python	[stress] msg_0013 @ 2026-02-25T18:16:20.8	sent	2026-02-25 18:16:20.955
33	Python	Java	[stress] msg_0014 @ 2026-02-25T18:16:20.9	sent	2026-02-25 18:16:21.054
34	Java	Python	[stress] msg_0014 @ 2026-02-25T18:16:20.9	sent	2026-02-25 18:16:21.060
36	Python	Java	[stress] msg_0015 @ 2026-02-25T18:16:21.0	sent	2026-02-25 18:16:21.163
35	Java	Python	[stress] msg_0015 @ 2026-02-25T18:16:21.0	sent	2026-02-25 18:16:21.164
37	Python	Java	[stress] msg_0016 @ 2026-02-25T18:16:21.1	sent	2026-02-25 18:16:21.266
38	Java	Python	[stress] msg_0016 @ 2026-02-25T18:16:21.1	sent	2026-02-25 18:16:21.270
39	Python	Java	[stress] msg_0017 @ 2026-02-25T18:16:21.1	sent	2026-02-25 18:16:21.379
40	Java	Python	[stress] msg_0017 @ 2026-02-25T18:16:21.1	sent	2026-02-25 18:16:21.384
41	Python	Java	[stress] msg_0018 @ 2026-02-25T18:16:21.1	sent	2026-02-25 18:16:21.510

3. Botón para la prueba de estrés el cual puede enviarse la cantidad deseada de mensajes



4. Botón para reconectar con PostgreSQL

ChatPyssage Python <-> Java | RabbitMQ + PostgreSQL

2026-03-13 19:52:46

ChatPyssage

Python <-> Java | RabbitMQ

CONTACTO

JB Java Backend
en línea

CONEXIONES

- PostgreSQL **conectado**
- RabbitMQ **conectado**

METRICAS EN VIVO

Enviados	1
Recibidos	0
Errores	0
Lat. avg (ms)	0
Lat. p95 (ms)	0

Ver historial BD

Prueba de estres

Reconectar

Java Backend RabbitMQ PostgreSQL

Java Java

Listo ya funciona
19:52:43 2026-03-01 18:30:06

Java Java

No me lleo el anterior
19:52:43 2026-03-01 18:49:37

No me envía a Java
19:52:43 2026-03-01 17:47:23

Creo que ya está funcionando
19:52:43 2026-03-01 18:25:01

Listo cambio de nombre
19:52:43 2026-03-01 18:48:51

Finalizado
19:52:43 2026-03-01 19:00:04

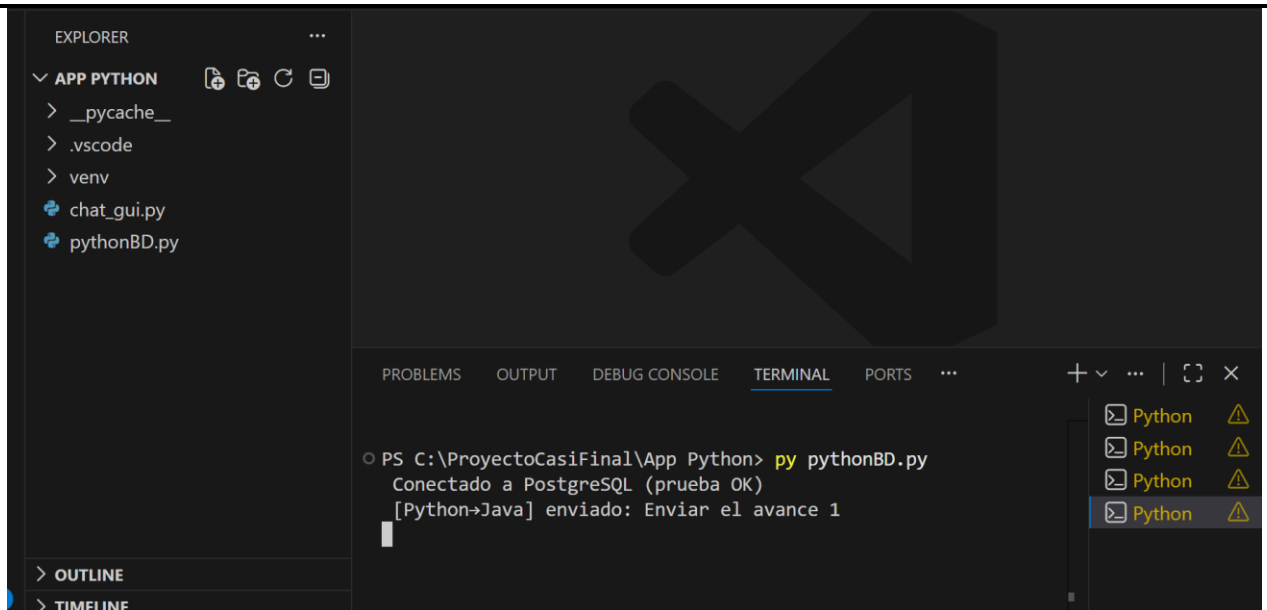
Enviar el avance 1
19:52:43 2026-03-13 19:03:36

Consumidor RabbitMQ activo - escuchando mensajes de Java

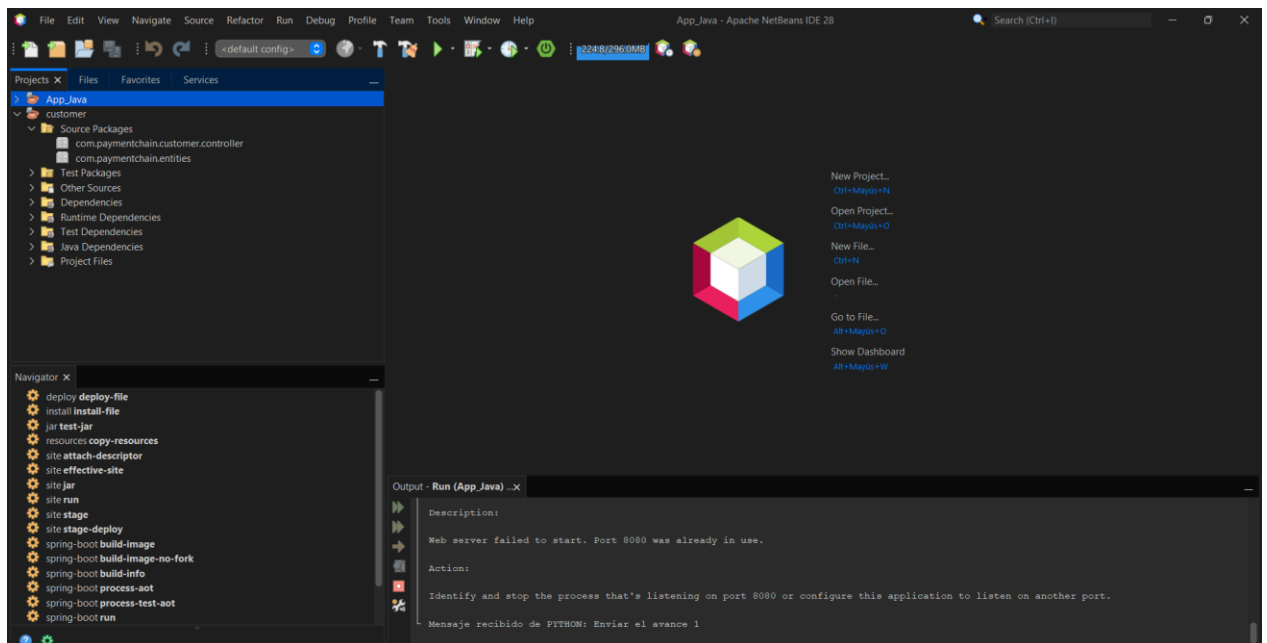
Enviar ->

Enter = enviar | Shift+Enter = nueva linea

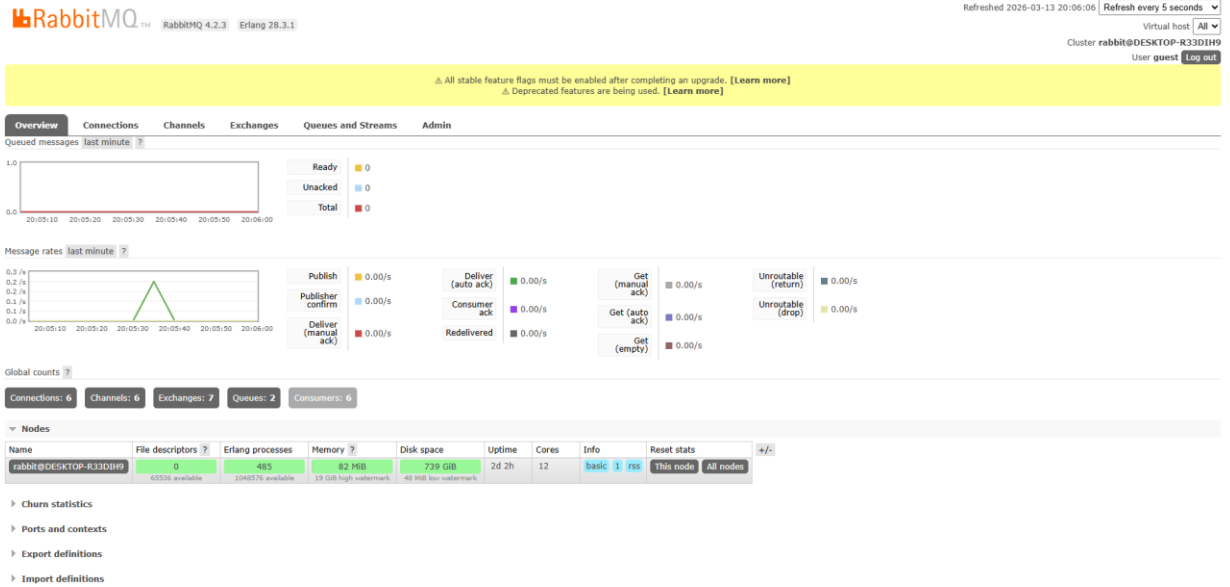
5. Desde Visual Studio se muestra el mensaje enviado



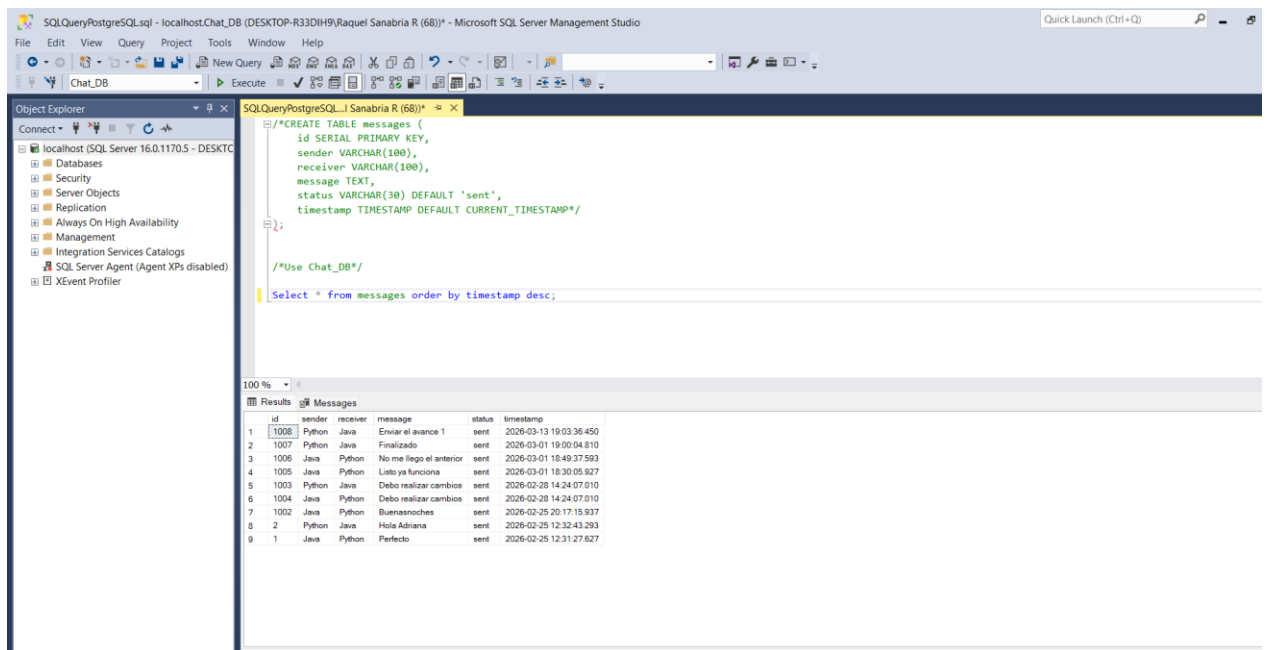
6. En ApacheNetBeans



7. RabbitMQ



8. Base de datos en SQL



SQLQuery1.sql - loc...el Sanabria R (53)) SQLQueryPostgresSQL...I Sanabria R (68))

```

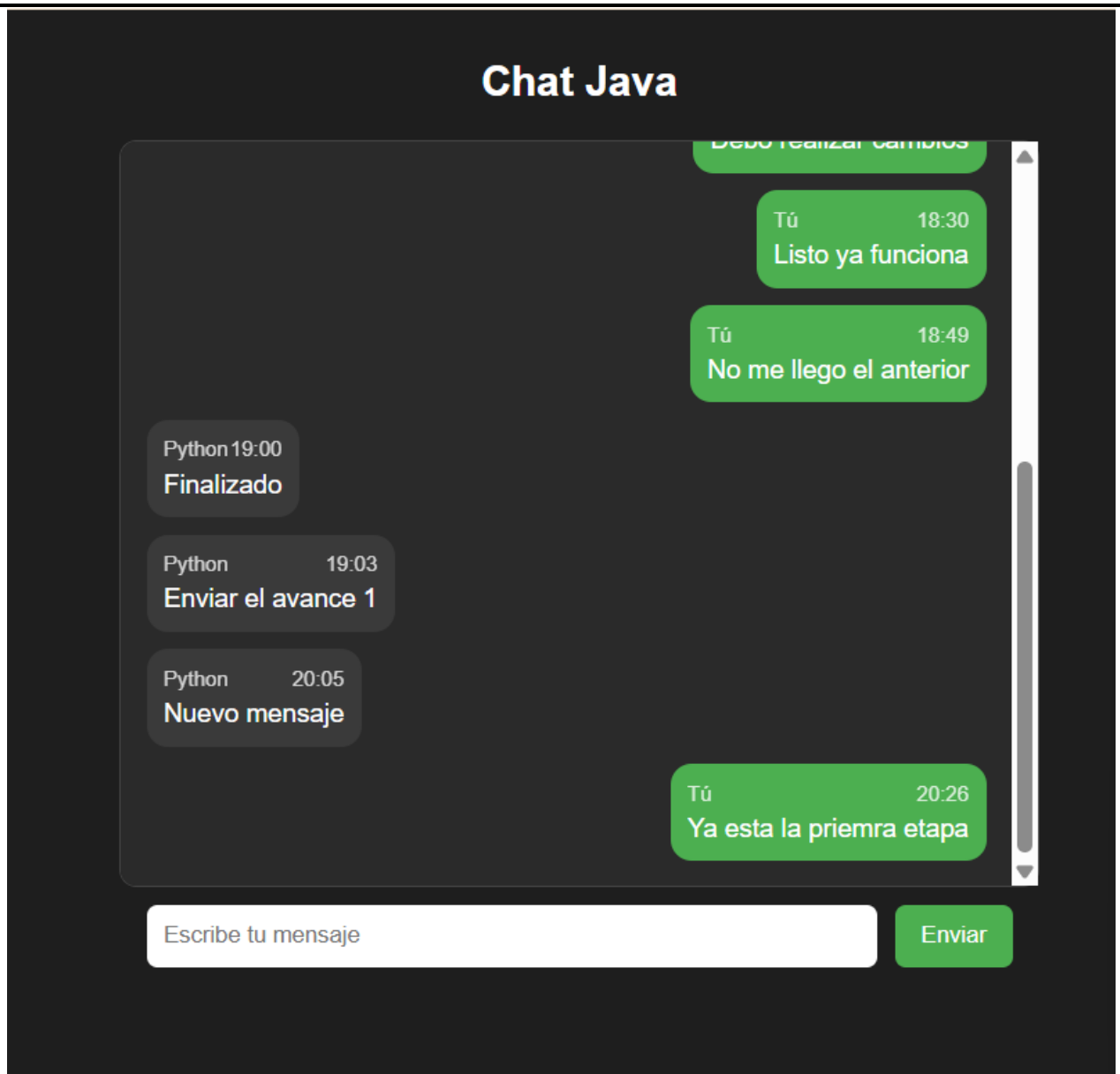
SELECT TOP (1000) [id]
      , [sender]
      , [receiver]
      , [message]
      , [status]
      , [timestamp]
FROM [Chat_DB].[dbo].[messages]
  
```

100 %

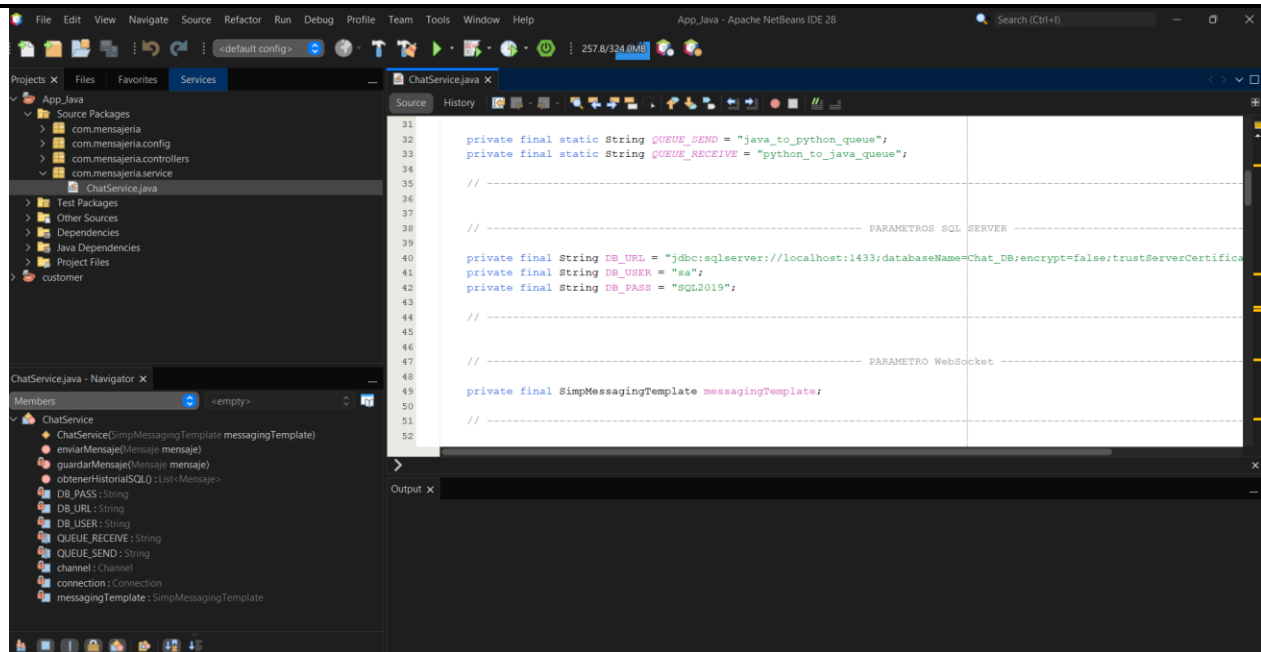
Results Messages

	id	sender	receiver	message	status	timestamp
1	1	Java	Python	Perfecto	sent	2026-02-25 12:31:27.627
2	2	Python	Java	Hola Adriana	sent	2026-02-25 12:32:43.293
3	1002	Java	Python	Buenasnoches	sent	2026-02-25 20:17:15.937
4	1003	Python	Java	Debo realizar cambios	sent	2026-02-28 14:24:07.010
5	1004	Java	Python	Debo realizar cambios	sent	2026-02-28 14:24:07.010
6	1005	Java	Python	Listo ya funciona	sent	2026-03-01 18:30:05.927
7	1006	Java	Python	No me lleo el anterior	sent	2026-03-01 18:49:37.593
8	1007	Python	Java	Finalizado	sent	2026-03-01 19:00:04.810
9	1008	Python	Java	Enviar el avance 1	sent	2026-03-13 19:03:36.450

9. Mensajes enviados desde Java



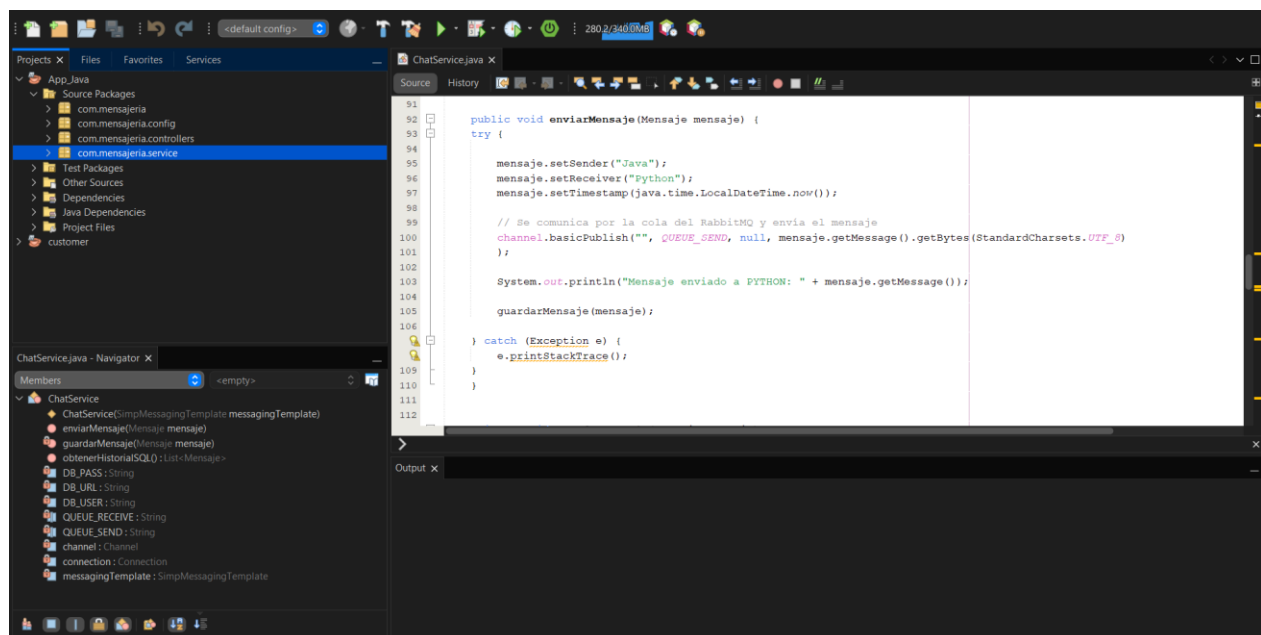
Código de Java



```

31 private final static String QUEUE_SEND = "java_to_python_queue";
32 private final static String QUEUE_RECEIVE = "python_to_java_queue";
33
34 // -----
35
36 // ----- PARAMETROS SQL SERVER -----
37
38 private final String DB_URL = "jdbc:sqlserver://localhost:1433;databaseName=Chat_DB;encrypt=false;trustServerCertificate=true";
39 private final String DB_USER = "sa";
40 private final String DB_PASS = "SQL2019";
41
42 // -----
43
44 // ----- PARAMETRO WebSocket -----
45
46 private final SimpMessagingTemplate messagingTemplate;
47
48 // -----
49
50
51
52

```

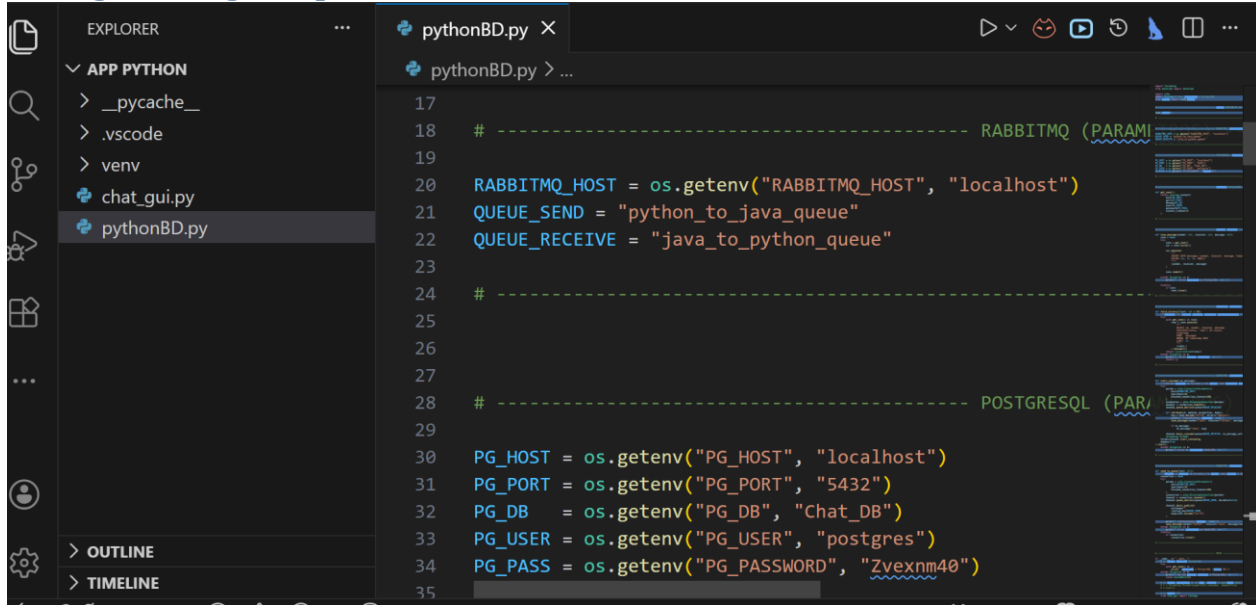


```

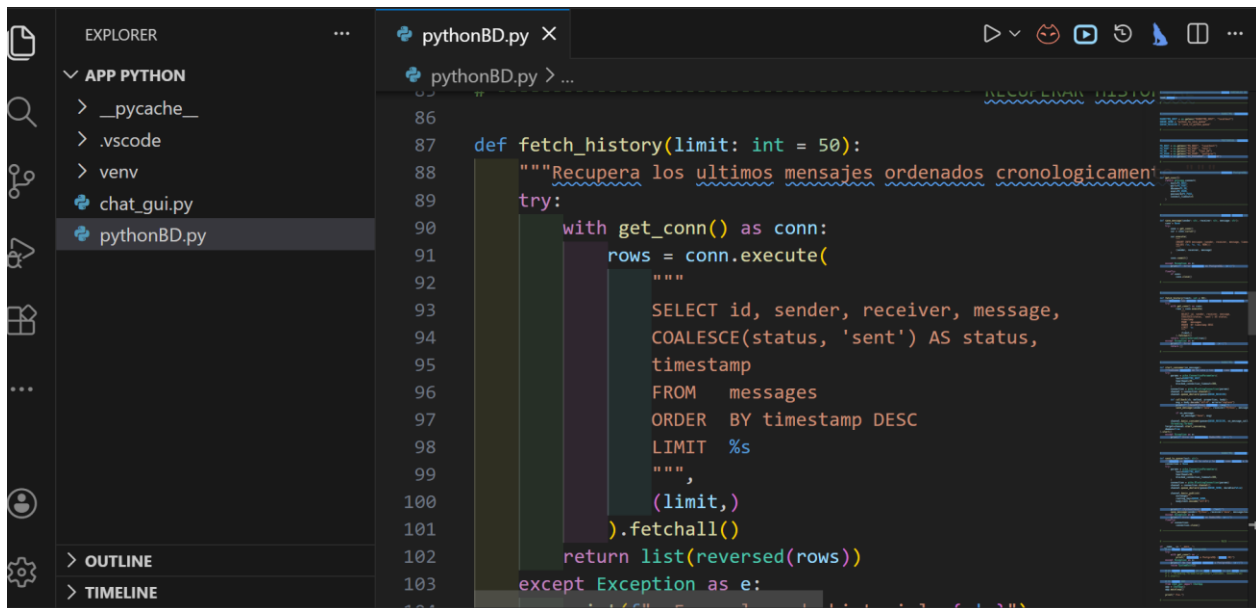
91
92 public void enviarMensaje(Mensaje mensaje) {
93     try {
94
95         mensaje.setSender("Java");
96         mensaje.setReceiver("Python");
97         mensaje.setTimestamp(java.time.LocalDateTime.now());
98
99         // Se comunica por la cola del RabbitMQ y envia el mensaje
100         channel.basicPublish("", QUEUE_SEND, null, mensaje.getMessage().getBytes(StandardCharsets.UTF_8));
101     }
102
103     System.out.println("Mensaje enviado a PYTHON: " + mensaje.getMessage());
104
105     guardarMensaje(mensaje);
106
107 }
108
109 catch (Exception e) {
110     e.printStackTrace();
111 }
112

```

Código PostgreSQL



```
17
18 # ----- RABBITMQ (PARAM)
19
20 RABBITMQ_HOST = os.getenv("RABBITMQ_HOST", "localhost")
21 QUEUE_SEND = "python_to_java_queue"
22 QUEUE_RECEIVE = "java_to_python_queue"
23
24 # -----
25
26
27
28 # ----- POSTGRESQL (PARAM)
29
30 PG_HOST = os.getenv("PG_HOST", "localhost")
31 PG_PORT = os.getenv("PG_PORT", "5432")
32 PG_DB = os.getenv("PG_DB", "Chat_DB")
33 PG_USER = os.getenv("PG_USER", "postgres")
34 PG_PASS = os.getenv("PG_PASSWORD", "Zvexnm40")
35
```



```
86
87 def fetch_history(limit: int = 50):
88     """Recupera los últimos mensajes ordenados cronológicamente"""
89     try:
90         with get_conn() as conn:
91             rows = conn.execute(
92                 """
93                 SELECT id, sender, receiver, message,
94                 COALESCE(status, 'sent') AS status,
95                 timestamp
96                 FROM messages
97                 ORDER BY timestamp DESC
98                 LIMIT %s
99                 """,
100                 (limit,)
101             ).fetchall()
102             return list(reversed(rows))
103     except Exception as e:
```